Component Specifics (Stage 3)

The components of the system are implemented as stateless Java services. Each API handler and the Task Assignment Engine run as multiple parallel instances behind a load balancer, and coordinate with each other only through the shared databases. The databases themselves (Task Database, Worker Database, Task Dependency Graph) are existing external systems that handle their own internal consistency and replication; the distributed systems concerns addressed in this stage are the ones that arise above the database layer.

This section enumerates, for each component, the request types it handles and the steps on the happy path. Where the handler relies on a database-level atomic operation, typically a conditional update or an atomic upsert, to coordinate with its parallel peers or with other workflows, this is noted explicitly. These atomic operations are the only shared-memory primitive the application layer has, and nearly every coordination concern in the system is resolved with one of them.

Task Input API Handler

A Task Input API Handler deals with three types of requests made by Task Inputters:

1. View current task data
    - Content of request: Task Inputter’s client ID, optional filters.
    - Desired response: list of current tasks with their metadata and dependency data.
2. Submit new task
    - Content of request: client ID, task data (name, description, deadline, priority, team, estimated time to complete, and optionally required skills and any dependency relationships). (Priority and estimated time to complete should have a few broad categories to prevent priority escalation and underestimation of time to complete.)
    - Desired response: confirmation of task creation.
3. Update existing task

    - Content of request: client ID, task ID, updated fields, expected version number.
    - Desired response: confirmation of update, or a conflict notification.

When a view request is made, the following happens:

1. A Task Input API Handler instance receives the request.
2. The handler queries the Task Database for the requested tasks and their metadata.
3. The handler queries the Task Dependency Graph for a view of the corresponding dependency edges.
4. The handler returns the data to the Task Inputter.

When a submit request is made, the following happens:

1. A Task Input API Handler instance receives the request.
2. The handler writes the task to the Task Database with a status of ‘unassigned’, using a conditional insert keyed on the task(’s hash??). If an equivalent task already exists, possibly because the Task Inputter is retrying a submission that succeeded but whose confirmation was lost, the insert is a no-op and the existing record is returned.
3. The handler returns a success message to the Task Inputter.

Delayed visibility of dependency edges: because the Dependency Handler propagates edges asynchronously, a “view” request made immediately after a “submit” may see the new task but not its dependency edges. This is the accepted consistency trade-off between the Task Database and the Task Dependency Graph.

When an update request for a task is made, the following happens:

1. A Task Input API Handler instance receives the request.
2. The handler applies the update to the Task Database using a conditional write that succeeds only if the record’s current version number matches the expected version. A successful write increments the version.

3. If the conditional write succeeds, the handler sends a confirmation. If it fails (because a concurrent update has modified the task since the Task Inputter last read it), the handler returns a conflict notification along with the task's current state, so that the Task Inputter can merge their changes and retry.

Worker API Handler

A Worker API Handler deals with three types of requests made by Workers:

1. Assign next task
    - Content of request: worker ID.
    - Desired response: the Worker's next assigned task, or a notification that no tasks are currently available.
2. Check current task
    - Content of request: worker ID.
    - Desired response: the Worker's currently assigned task, or a notification that no task is currently assigned.
3. Report task completion
    - Content of request: worker ID, task ID.
    - Desired response: confirmation of task completion.

When an assign request is made, the following happens:

1. A Worker API Handler instance receives the request.
2. The handler reads the Worker's state from the Worker Database.
    a. If the state is 'assigned': the handler reads the Worker's current assignment from the Worker Database and returns it directly (skipping step 3), without invoking a Task Assignment Engine instance.
    b. If the state is 'in progress' and has not timed out given the current local time of the Worker Database server: the handler notifies the Worker that it is waiting and then re-evaluates from step 2.
    c. If the state is 'open' or if the state is 'in progress' but has timed out given the current local time of the Worker Database server: the handler performs a conditional update on the Worker's state from 'open' to 'in progress' with

the current local time of the Worker Database server for timeout purposes. If the update fails (because a parallel instance has already performed the same transition), the handler re-evaluates from step 2. If it succeeds, the handler contacts a Task Assignment Engine instance to assign a new task.

Note: the ‘in progress’ timeout is a design trade-off. It must be long enough that a slow but functioning Task Assignment Engine is never interrupted prematurely, and short enough that Workers stuck behind a crashed instance are unblocked promptly.

3. The engine returns either the assigned task or a notification that no tasks are available to the handler.
4. The handler returns the result to the Worker.

When a check request is made, the following happens:

1. A Worker API Handler instance receives the request.
2. The handler queries the Worker Database for the Worker’s current assignment and state.
3. The handler returns the current assignment if the Worker’s state is ‘assigned’ (or a notification of none) to the Worker.

When a completion request is made, the following happens:

1. A Worker API Handler instance receives the completion request from a Worker.
2. The handler reads the Worker Database to verify that the Worker's state is ‘assigned’ and that the submitted task ID matches the Worker's current assignment.
3. The handler marks the task as ‘completed’ in the Task Database using a conditional update that succeeds only if the task is currently recorded as assigned to this Worker. If the update fails (because an Admin is in the midst of changing the state), the handler returns a conflict notification (skipping step 4).
4. The handler clears the Worker’s current assignment and sets the Worker’s state to ‘open’ in the Worker Database using a conditional update that succeeds only if the Worker's current assignment is still this task and the Worker’s state is

‘assigned’. If the update fails, the handler attempts to clear the Worker's current assignment using a conditional update that succeeds only if the Worker's current assignment is still this task (this should not succeed but exists to cover unforeseen edge cases). If the second update also fails (because an Admin is in the midst of changing the state), the handler returns a conflict notification.

5. The handler returns a confirmation or conflict notification to the Worker.

Task Assignment Engine

A Task Assignment Engine deals with one type of request made by instances of the Worker API Handler:

1. Assign Worker’s next task
    - Content of request: worker ID.
    - Desired response: the newly assigned task, or a notification that no tasks are currently available.

When an assign request is made, the following happens:

1. A Task Assignment Engine instance receives the assignment request from a Worker API Handler.
2. The engine queries the Task Dependency Graph for all currently incomplete and unblocked tasks.

   Note: a task whose last dependency was just marked ‘completed’ in the Task Database may not yet appear as unblocked in the graph until the Dependency Handler's next polling cycle. In such a case, the engine will assign a less optimal task. If there are no other available tasks, it will return a notice that no tasks are currently available.
3. The engine queries the Task Database for the metadata of the tasks in the unblocked task list.
4. The engine examines the data to filter out any task that has already been assigned or completed. During the examination, if it finds an incomplete task already recorded as assigned to this Worker (because a prior assignment attempt was interrupted after being written to the Task Database but before being

written to the Worker Database write), the engine (skipping steps 5-8) attempts to record the assignment in the Worker Database and set the Worker's state to ‘assigned’ using a conditional update that succeeds only if the Worker’s state is not ‘assigned’ or the Worker’s task is this task. If the update succeeds, the engine returns the task. If it fails (because of a conflict with a though-to-be-timed-out engine instance), the engine unassigns the task from this Worker in the Task Database.

Note: the engine may fail after assigning the task in the Task Database but before assigning it to the Worker in the Worker Database. This is okay because the Worker will eventually request a new task again. The Task Dependency Graph is assignment-agnostic and so will still return that previously-assigned task in its list of tasks. In order to guard against the possibility that a new task may have superseded that one in priority in the meantime, which would mean that the engine won't look at the previously-assigned task and miss it, the Task Assignment Engine checks all the tasks it receives from the Task Dependency Graph to see if they are already owned by this Worker before trying to assign a new one. This won’t help if the Worker stops working and never again requests a new task, but that edge case is an acknowledged risk and can be handled by Administrators.

5. The engine queries the Worker Database for the Worker’s metadata (team, skills, etc.).
6. The engine filters the unblocked task list to tasks eligible for this Worker.
7. The engine sorts the task list by various factors, including priority, deadline, and estimated time to complete.
8. The engine iterates through the sorted list. For each candidate task, the engine attempts a conditional update in the Task Database to change the task’s state from ‘unassigned’ to ‘assigned’ and enter this Worker as the assigned Worker. If the update fails (because another engine instance has claimed it), the engine advances to the next candidate.

Note: if the Employee Data API Handler updates the Worker's skills between the engine's read in step 5 and the commit in step 8, the engine may assign a task that no longer matches the Worker's data.

9. On the first successful conditional update, the engine records the assignment in the Worker Database and sets the Worker's state to 'assigned' using a conditional update that succeeds only if the Worker's state is not 'assigned' or the Worker's task is this task. If the update succeeds, the engine returns the task. If it fails (because of a conflict with a though-to-be-timed-out engine instance), the engine unassigns the task from this Worker in the Task Database.
10. The engine returns the assigned task to the Worker API Handler.
    Note: if the engine exhausts the candidate list in step 8 without any successful update, it reverts the Worker's state to 'open' and returns a notification that no tasks are currently available.

Admin API Handler

An Admin API Handler deals with many types of requests made by Administrators. Here are two:

1. View data
    - Content of request: admin ID, optional filters.
    - Desired response: the requested task and/or worker data.
2. Assign a task to a Worker
    - Content of request: admin ID, task ID, target worker ID.
    - Desired response: confirmation of assignment.
      Note: this request is singled out because it is the primary administrative override, used e.g. to reopen tasks incorrectly marked as completed or to handle the Task Assignment Engine failure edge case mentioned in the note following step 3 of the assign request workflow of the Task Assignment Engine.

When a view request is made, the following happens:

1. An Admin API Handler instance receives the request.

2. The handler queries the relevant database(s) for the relevant data.
3. The handler returns the data to the Administrator.

When an assign request is made, the following happens:

1. An Admin API Handler instance receives the request.
2. The handler reads the task's state from the Task Database.
    a. If the state is ‘open’ or ‘completed’: the handler performs a conditional update on the task's state from ‘open’ or ‘completed’ to ‘held by Admin’. If the update fails (because it has been assigned to a Worker), the handler re-evaluates from step 2.
    b. If the state is ‘assigned’: the handler performs a conditional update on the task's state from ‘assigned’ to ‘held by Admin’ (but does not clear the task’s assigned Worker). If the update fails (because it is no longer assigned), the handler notifies the Administrator that the task is actually not currently assigned and re-evaluates from step 2. If it succeeds, the handler clears the Worker’s current assignment and sets the Worker’s state to ‘open’ in the Worker Database using a conditional update that succeeds only if the Worker's current assignment is still this task and the Worker’s state is ‘assigned’. The handler then clears the task’s assigned Worker in the Task Database using a conditional update that succeeds only if the task’s state is still ‘held by Admin’ and the assigned Worker is this Worker.
    c. If the state is ‘held by Admin’: the handler notifies the Administrator that the task is currently held by an Administrator and then re-evaluates from step 2.

    Note: in between step 2 and step 3 the task has a state of 'held by Admin' with no corresponding Worker lock. There is no automatic timeout for this state in case of failure. An Administrator can query the Task Database for tasks with the state 'held by Admin' and either resume the reassignment or restore the previous state.
3. The handler reads the target Worker's state from the Worker Database:

a. If the state is ‘open’ or ‘assigned’: the handler performs a conditional update of the state from ‘open’ or ‘assigned’ to ‘in progress’ without setting a timeout time for ‘in progress’ (unlike step 2c of the assign request workflow of the Worker API Handler) (but does not clear the Worker’s assigned task if it was assigned). If the state was ‘assigned’, the handler clears the formerly assigned task’s state in the Task Database using a conditional update that succeeds only if the task’s state is still ‘assigned’ and its assigned Worker is this Worker.
   Note: at one point in this step the target Worker is locked (without a timeout) and their task is stuck assigned to them. As noted earlier, an Administrator can inspect and fix the state by either resuming the reassignment or restoring the previous state.
b. If the state is ‘in progress’: the handler notifies the Administrator that it is waiting and then re-evaluates from step 3.

4. The handler performs a conditional update of the requested task’s state in the Task Database from ‘held by Admin’ to ‘assigned’ and assigns it this Worker.
5. The handler records the assignment in the Worker Database.
6. The handler sends a confirmation to the Administrator.
   Note: this workflow does not have automatic partial-failure recovery. This is an acceptable trade-off because Administrator operations should be infrequent and they inherently involve a human. The conditional updates at each step prevent the system from reaching a silently inconsistent state; what remains after a failure are well-defined "stuck" states ('held by Admin' task, locked Worker) that an Administrator can query and resolve.

Employee Data API Handler

The Employee Data API Handler deals with one type of request made by the External Employee Database:

1. Submit new data

    - Content of request: employee ID, metadata (name, team, skills), source timestamp.
    - Desired response: confirmation of update.

When a submit request is made, the following happens:

1. The Employee Data API Handler receives the request.
2. The handler applies the update to the Worker Database as an upsert: if no record exists for this employee, one is created with a state of ‘open’; otherwise, the existing record's metadata fields are replaced in full. The upsert checks the source timestamp against the record's last-updated timestamp, and ignores updates that are older.
3. The handler sends a confirmation to the External Employee Database.

Dependency Handler

The Dependency Handler is a background polling process that monitors the Task Database for changes requiring propagation to the Task Dependency Graph. It does not serve requests from external parties.

Each row in the Task Database carries a monotonically increasing sequence number (change_seq) assigned by the database on every insert or update. The Dependency Handler persists a single cursor: the highest change_seq it has successfully processed. Each polling cycle is therefore a single indexed range scan — it fetches only rows that are new since the last cycle — rather than a full-table scan. On restart after a crash, the handler re-reads the persisted cursor and resumes from where it left off.

Polling cycle:

1. The handler queries the Task Database for all rows whose change_seq is greater than the stored cursor, ordered by change_seq ascending.
2. For each returned row, the handler inspects what changed and propagates the change to the Task Dependency Graph:
    a. If the row has dependency edges not yet reflected in the graph, the handler writes those edges to the graph.

b. If the row’s state is now ‘completed’ and the graph shows the task as incomplete, the handler marks the node as ‘completed’ and re-evaluates its dependents, unblocking any whose remaining dependencies are all now ‘completed’.
c. If a formerly ‘completed’ row’s state is no longer ‘completed' (because an Administrator has moved the task from ‘completed’ to ‘held by Admin’) and the graph shows it as 'completed', the handler marks the node as incomplete and re-evaluates its dependents, re-blocking any whose dependencies are no longer all ‘completed’.

3. The handler advances the stored cursor to the highest change_seq processed in this cycle.
4. The handler waits for the next polling interval.
   Note: changes are visible in the Task Database immediately but take up to one polling interval to reach the Task Dependency Graph. During this window the Task Assignment Engine may read stale data from the graph. This is the same availability-over-consistency trade-off accepted throughout the system, and is bounded by the polling interval.